

PYTHON PROGRAMMING

Final Exam

Example

Points

	LO1	LO2	LO3	LO4	LO5	LO6	LO7	LO8	TOTAL
Points	10	10	5	15	12	8	8	12	80
Time	20 min	20 min	10 min	30 min	24 min	16 min	16 min	24 min	160 min

INSTRUCTIONS:

- Only IDLE Shell is allowed to be used for this exam.
- Submit a separate file for each learning outcome.
- The written program must successfully start (without errors), otherwise the task will be graded with 0 points.
- Each time you read from the keyboard; you must display to the user what is expected. For example, if you are reading a first name, you must display "Enter a first name:" to the user.

The use of any equipment, services or products not explicitly stated in the instructions is not permitted during the exam.

Learning outcome 1 – 10 points / 20 min

1. [LO1_M / 7 points]

Write a program that will ask the user to enter information about five authors, five songs and five song durations in seconds. Determine the number of minutes and seconds each song lasts and print them at the end of the program.

2. [LO1_D / 3 points]

Ask the user to enter a sign that will be used to indicate the length of songs with the character "*", and then display a "graphical" representation of lengths of songs in minutes and seconds, for example:

Author: Ananda Lies, song: Lay Cold
 Minutes: *** (3)
 Seconds: ***** (16)
 Author: Jelena Radan, song: SALUT!
 Minutes: **** (4)
 Seconds: ***** (13)
 Author: Luka Nižetić, song: Južina
 Minutes: *** (3)
 Seconds: ***** (10)
 Author: Ogenj, song: Daj, daj
 Minutes: ***** (5)
 Seconds: ***** (6)
 Author: Supersonic Trio, song: Animal
 Minutes: *** (3)
 Seconds: ***** (25)

Learning outcome 2 – 10 points / 20 min

1. [LO2_M / 7 points]

Write a program that will ask the user to enter information about five authors, five songs and five song durations in seconds. Determine how many songs have a length between three and four minutes, find the longest and the shortest song. Print the shortest and the longest song with the authors and durations to the console and print the next songs that are the nearest to the shortest and the longest song by the length. Only “if-else” statements are allowed. You shouldn’t use collections like lists, sets, dictionaries, and tuples.

2. [LO2_D / 3 points]

Extend the program described in the previous assignment with an additional segment that determines the song with the longest name. Print the artist, song name and the duration of the song with the longest name to the console. Only “if-else” statements are allowed. You shouldn’t use collections like lists, sets, dictionaries, and tuples.

Learning outcome 3 – 5 points / 10 min

1. [LO3_M / 3 points]

Write a program that will ask the user to enter information about five authors, five songs and generate random numbers of seconds that represent the length of the song. By using an appropriate collection, find the song with the shortest and the longest name and print the author, song name and the duration in minutes and seconds of those songs to the console.

2. [LO3_D / 2 points]

Extend the program described in the previous assignment with two functions: the first returns the collection of songs gathered from the user, and the second that determines and returns the data about the shortest and the longest song. Call the functions from the main program and print out the final output values.

Learning outcome 4 – 15 points / 30 min

1. [LO4_M / 10 points]

Write a Song class with:

- Attribute “name” (string) initialized in constructor.
- Attribute “genre” (string) initialized in constructor.
- Attribute “length in seconds” (integer) initialized in constructor.
- String representation method in format: Vehicle {name} {genre} – {length in seconds}
- Method “convertLengthToMinutesAndSeconds” that return the length of the song in seconds.

Write a program that asks the user to input data needed to create five Song objects and add them to a list. Create a method that takes the list of Song objects and returns the song with the shortest length. If two songs are the same length, you need to sort them by name (alphabetically). Print the name, genre and length of the song with shortest length in minutes and second.

2. [LO4_D / 5 points]

Create two subclasses of the Song class: PopSong and ClassicalSong. The PopSong adds new variables “artist” (string), “isBillboardHit” (boolean) and “albumName” (string) to a class and extends the constructor with them. The ClassicalSong add new variables “composer” (string), “orchestraName” (string) and “movementNumber” (integer) to a class and extends the constructor with them. Update the program from the previous assignment with a new method

that asks user to input data needed to create three PopSong and three ClassicalSong objects, adds them to the **same** list and returns the list. Print all data from PopSong and ClassicalSong objects after the function call and determine all the pop songs that are billboard songs, the classical song with the lowest movement number.

Learning outcome 5 – 12 points / 24 min

1. [LO5_M / 10 points]

Implement a function that reads the textual file with the following content (author name, song name, length in minutes, the remainder of the length in seconds):

Ananda Lies, Lay Cold, 3, 16
 Jelena Radan, SALUT!, 4, 13
 Luka Nižetić, Južina, 3, 10
 Ogenj, Daj daj, 5, 6
 Supersonic Trio, Animal, 5, 6

and returns sorted list by total length in seconds. Call the function from the main program and print the results to the console.

2. [LO5_D / 2 points]

Write a second function that serializes the data related to the song with the shortest length to a JSON file called "theShortestSong.json". Call the function from the main program.

Write a third function that serializes all the songs with longest length to a file "theLongestSongs.json". If two songs with the same longest length exist (like "Ogenj, Daj daj" and "Supersonic Tri, Animal") serialize them all to the file. Call the function from the main program.

Learning outcome 6 – 8 points / 16 mins

1. [LO6_M / 6 points]

Write tests that ensure that the following functions works properly:

```
# Function to create a generic song
def create_song(name, genre, length_in_seconds):
    return {"name": name, "genre": genre, "length_in_seconds": length_in_seconds}

# Function to create a pop song
def create_pop_song(name, artist, album, length_in_seconds):
    return {"name": name, "genre": "Pop", "artist": artist, "album": album, "length_in_seconds": length_in_seconds}

# Function to create a classical song
def create_classical_song(name, composer, movement_number, length_in_seconds):
    return {"name": name, "genre": "Classical", "composer": composer, "movement_number": movement_number, "length_in_seconds": length_in_seconds}

# Function to get song description
def get_description(song):
    if song["genre"] == "Pop":
        return f'"{song["name"]}" by {song["artist"]} from the album "{song["album"]}" is a pop song that lasts {song["length_in_seconds"]} seconds.'
    elif song["genre"] == "Classical":
```

```
    return f"""{song['name']}', movement {song['movement_number']}, composed by {song['composer']}, is a classical
    piece that lasts {song['length_in_seconds']} seconds."
```

```
else:
```

```
    return f"""{song['name']}' is a {song['genre']} song that lasts {song['length_in_seconds']} seconds."
```

The tests must ensure that the functions works correctly: the “get_description” must generate a valid description for “generic”, “pop” and “classical” song and the length of the song in seconds in calculated properly.

2. [LO6_D / 2 points]

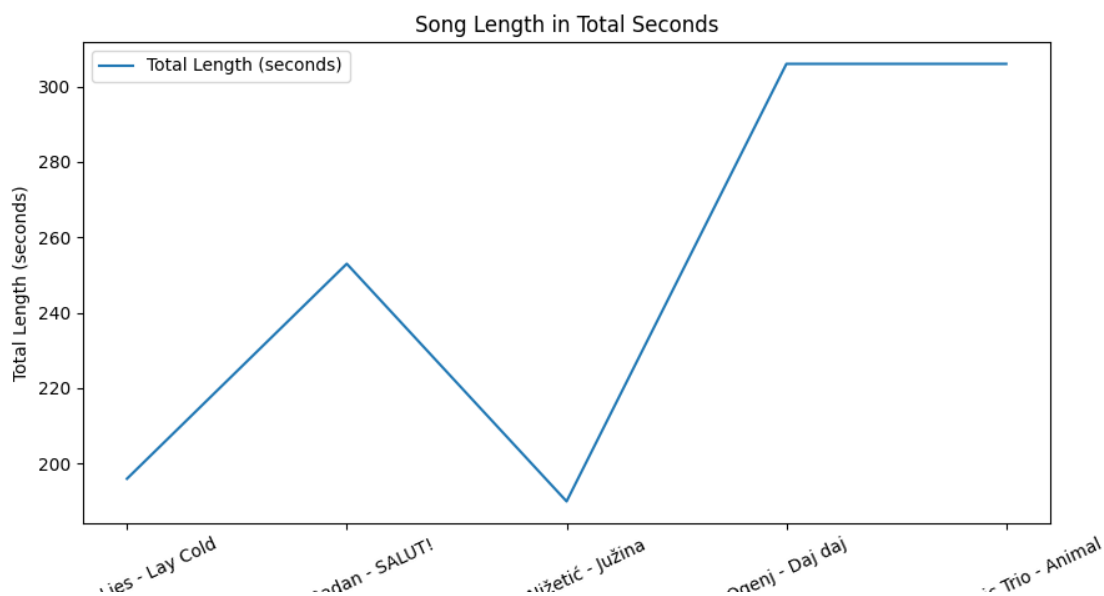
Update the functions definitions to throw an error in case of a negative value for the length of the song and write an unit test that checks if the error was thrown properly.

Learning outcome 7 – 8 points / 16 mins

1. [LO7_M / 6 points]

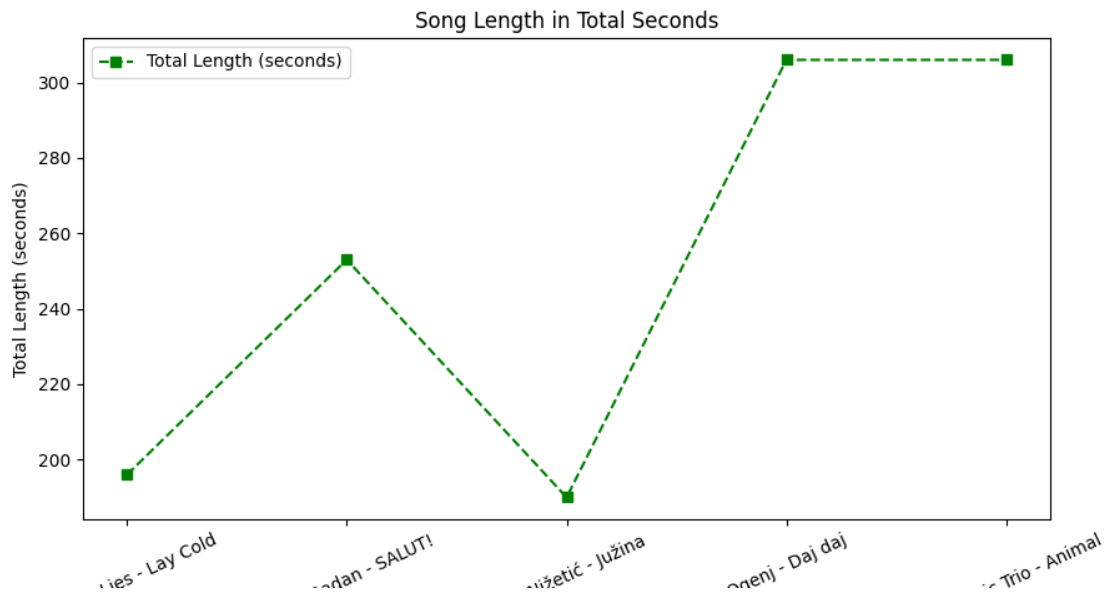
By using the matplotlib library write a program that plots the following chart based on the following list of values (the two numbers represent the minutes and seconds – the chart contains the lengths in seconds):

```
songs = [
    ("Ananda Lies - Lay Cold", 3, 16),
    ("Jelena Radan - SALUT!", 4, 13),
    ("Luka Nižetić - Južina", 3, 10),
    ("Ogenj - Daj daj", 5, 6),
    ("Supersonic Trio - Animal", 5, 6),
]
```



3. [LO7_D / 2 points]

Update the chart with additional settings to display the following additional details to the chart:



Learning outcome 8 – 12 points / 24 mins

1. [LO8_M / 10 points]

Use the REST API from Radio Basilisk that contains the information about the last 20 songs that were played by using the URL <https://radio-basilisk.api.radiosphere.io/channels/87d0aaab-1db2-453b-af40-63bdde8cd1d1/track-history>. Print all 20 artists and song names played, for example:

Alex Warren – Carry You Home

etc. for the current JSON format:

```
items": [
  {
    "track": {
      "trackId": "39fe9027-0736-4014-8b0d-cd8d3b372c02",
      "title": "Carry You Home",
      "artistCredits": "Alex Warren",
      "artwork": "https://cdn.radiosphere.io/images/8026f9c7-56c3-5598-8fd8-c8738c779d08"
    },
    "start": "2025-02-09T12:32:37.337Z"
  },
  etc.
```

Use the HTML web page <https://www.songfacts.com/search/> to get the information about the songs. For example, for the song „Carry You Home“ and the artists „Alex Warren“ fetch the HTML page and find the name of the album and print it to the console. The search page must be constructed like this:

<https://www.songfacts.com/facts/alex-warren/carry-you-home>

The fetched HTML page looks like this:

Carry You Home by Alex Warren



Album: You'll Be Alright, Kid (Chapter 1) (2024)

Charted:  23  104

[License This Song](#) 

The results must be printed in this format for all the songs:

title: Carry You Home
artist: Alex Warren
album: You'll be Alright, Kid (Chapter 1)

2. [LO8_D / 2 points]

Write a function that fetches the year of the album and "Charted" information like showed on the screenshot in the previous assignment (UK: 23, USA: 104) and returns is to the main program so the print to the console looks like this:

title: Carry You Home
artist: Alex Warren
album: You'll be Alright, Kid (Chapter 1) [2024]
Charted UK: 23
Charted USA: 104